# Chronicle: a technical write-up

**Abstract**

Chronicle is a hook-delivered memory layer for Claude Code that retrieves project context from prior session traces. This write-up describes its retrieval architecture, evaluation harness, and early results on vague, behavior, and integration tracks. The clean paired runs show the strongest gains in prior-context signal and project-specific answer quality; cost and wall-time deltas are smaller and still uncertain.

**Keywords:** coding agents; memory; retrieval; evaluation; Claude Code

I started digging into `MEMORY.md` after I spent a day constantly switching between 2 different Claude Code (CC) accounts, having to explain the same deployment process over and over again in new chats, wondering why on earth CC couldn't have some sort of memory level (I could have put these deployments into a .md file, that is my fault). What I found was a single markdown file. Capped at 200 lines or 25 KB, loaded at session start, written to by a prompt that decides what's worth remembering, and periodically pruned by something called "Auto Dream."

I opened mine. Quite literally, 1 line. This project had existed for around 2 weeks. I checked the other projects I'd been working on - same story. I actually do understand why it is so limited, everything feels important, but when you're prompting into every single prompt that can start to add up a bit.

My thought was, I've built RAG before, I've written parsers before. I can figure this out.

## 1 How Chronicle works

Chronicle is a per-prompt retrieval layer for Claude Code. It indexes the project's real session JSONLs (the transcripts CC writes to `~/.claude/projects/<encoded>/<uuid>.jsonl`) and writes a `<chronicle-context>` block into the model's view of the next turn. The substrate is the user's own prior sessions; nothing synthetic.

### 1.1 Two retrieval substrates

Chronicle runs two indexes in parallel.

**Substrate A - Conversation and action chunks.** The session JSONLs are split into ~400-word windows and embedded with `BAAI/bge-small-en-v1.5` (33M params, 384-d, ~6 ms encode steady state), then stored in sqlite + sqlite-vec. The spoken chunk path keeps user/assistant exchange and deliberately excludes `tool_use` markers, `tool_result` content, `thinking` blocks, slash-command echoes, and synthetic tool-result-as-user turns. The current chunker also creates `behavioral` chunks for standard Bash/Edit/Write/MultiEdit tool inputs and `tool_result` chunks for filtered failures or narrow deploy/publish success evidence.

The reasoning: a query about `make_cache_key` should match the discussion of the function, not a code dump that happens to contain it. Chunk IDs are content-addressed (`sha256(session || first_turn || content)[:16]`), so unchanged JSONLs re-index for free.

**Substrate B - Typed-claim memory store.** Markdown files extracted offline from the same JSONLs by Claude Haiku. Each file has metadata at the top - `kind`, `polarity`, `referent`, `source_session`, `source_turn` - and is indexed twice: FTS5 (BM25) for keyword and sqlite-vec for embedding, combined with Reciprocal Rank Fusion. A graph of replaced memories sits alongside, with links between old and new claims, plus cycle checks at construction time.

### 1.2 The execution loop

A `UserPromptSubmit` event with both substrates enabled currently runs through this shape:

1. `chronicle hook` reads the payload from stdin (prompt, session_id, project_dir, recent_turns, compaction_count).
2. A path-triage classifier (`SKIP / LIGHT / FULL`) inspects prompt shape. SKIP exits silently; LIGHT only adds session-state blocks.
3. Conversation search tries the daemon first. If the daemon is unavailable, the hook falls back to an in-process embedder and store.
4. On the non-composer conversation path, the index is refreshed by JSONL `mtime`. On the composer path, Chronicle searches an existing conversation index; incremental refresh there is still a limitation.
5. The query runs through dense, hybrid, or multi-query search depending on config and prompt shape, with `min_top_cosine`, rerank, and MMR controls when enabled.
6. Candidate chunks are re-ranked with polarity, topic overlap, recency, list-answer penalties, stale/replacement signals, and citation-use scores.
7. The typed-claim store is searched alongside the raw chunks; results use kind, polarity, supersession, temporal, and lexical scorers.
8. A routing-weights classifier (sub-millisecond, no LLM) outputs source and memory-kind weights based on prompt patterns.
9. The composer allocates source budgets, removes duplicate overlap, formats blocks, and writes a `<chronicle-context>` block to stdout.
10. Claude Code prepends the block to the model's view of the turn.

The hook always exits 0; failures inside `run_hook` are caught and logged. The 5-second timeout in `settings.json` is the safety net. Steady-state hot path is tens of milliseconds once the embedder is warm.

## 1.3 Hook events

The default installer wires only `UserPromptSubmit`. The full set is opt-in via `~/.claude/settings.json`:

**Table 1:** Claude Code hook events used by Chronicle.

| Event | Purpose |
|---|---|
| `UserPromptSubmit` | Front-of-turn injection for the changing per-prompt path |
| `SessionStart` | Stable prelude from cached conventions and session state |
| `PreToolUse:Read/Grep/Glob` | Mid-turn context injection for read/search tool results |
| `PreToolUse:Bash` | Action-time guard; normalises and classifies the command, then checks it against learned conventions |
| `PostToolUse:Read` | Annotates memory-file reads with replacement-graph status |
| `Stop` / `SubagentStop` | Citation grading on what the agent actually used |
| `SessionEnd` | Typed-claim extraction over the just-finished session |

## 1.4 Triage and the composer

The triage module is two related classifiers. Path triage (`SKIP / LIGHT / FULL`) decides whether retrieval runs at all. Routing weights decides how the substrates are blended:

**Table 2:** Triage and routing classifiers used by the composer.

| Sub-classifier | Output | Trigger |
|---|---|---|
| `classify_reference_scope` | `ReferenceScope` | "we already / last time / earlier you" -> PAST_SESSIONS; short imperative under 80 chars -> NONE |
| `classify_claim_type_weights` | {\{MemoryKind: float\}} | Convention/fix/constraint signals boost the corresponding kinds |
| `classify_memory_chunk_priority` | `(memory_w, chunk_w)` | File paths and `Class.Method` shapes favour chunks; abstract "what should we use" shapes favour memories |

The composer uses these weights to split the context budget, sort results from each source, remove overlap (a

chunk that is just a memory's source turn gets dropped under `prefer_memory`), and format block sections. It does not yet do one global weighted merge because memory scores and chunk scores live on different scales.

### 1.5 Supersession

The supersession graph is built from each memory's `revises` field. It exposes `is_superseded`, `superseded_by`, `latest_in_chain`, and a bi-temporal `validity_at(t)` projection inspired by Zep / Graphiti. The graph powers two retrieval behaviours: a `SUPERSEDED_PENALTY = 0.5` multiplicative downrank during scoring, and the `PostToolUse:Read` annotation that catches mid-turn reads of stale memory files even when they bypass the front-of-turn injection.

## 2 RAG has limits, and what remains

While I was building the first version of Chronicle, I always considered the idea of "frontier labs are moving away from RAG" as fact. That framing is too broad. The weaker pattern is single-shot retrieval that runs before the model has understood the task.

The useful points for Chronicle are narrower:

- Dense similarity can miss intent. Negation, time, and composition are the cases a memory system most needs to separate: `X` vs `not X`, or "we use Postgres" vs "we used to use Postgres".
- A pre-turn query is only an early guess. If it misses the right record, the model gets no signal that something is absent.
- Agentic search works well for current source code and live tools, but it still depends on the model deciding to search. Prior sessions are not in the repo, so there is often nothing obvious to grep.

The most relevant paper result is the memory-tool failure mode. MEMTRACK reports that agents do not reliably call memory tools, and that adding memory tools can increase redundant tool calls. Chronicle does not claim retrieval is solved. It tries to put prior context in view before the agent has to know it exists.

## 3 Chronicle's approach

Chronicle uses hooks instead of a model-chosen tool. On each `UserPromptSubmit`, it renders memory into the next turn, so the model does not have to ask for it. The hot path is local: embedding lookup, hybrid scoring, triage, and composition. There is no extra model call in the prompt path.

It targets a narrow blind spot: cross-session knowledge. The conversation index separates spoken exchange windows from tool/action evidence, and typed claims keep `source_session` and `source_turn`, so a retrieved claim can be checked against the transcript that produced it.

The tradeoff remains. Chronicle still embeds, so it inherits embedding limits. It still retrieves before the model thinks through the turn, so it can miss. Supersession only helps when old and new claims both make it into the candidate set. The current bet is modest: for a small, user-owned session history, hook delivery and source tracking are useful enough to test.

## 4 Design decisions

A few decisions carry most of the design.

**Hook delivery, not MCP.** MCP would make memory a tool the model chooses to call. Hooks put memory into the prompt path without asking the model to opt in, which matches the MEMTRACK failure case better. MCP also adds a round trip on the path where latency matters most.

**LLM-free hot path.** Per-prompt work runs locally: embedding, sqlite-vec lookup, heuristic re-rank, triage, and composition. `ANTHROPIC_API_KEY` is only needed for offline typed-claim extraction and optional graders/planners.

**BGE-small over MiniLM.** The choice was a tradeoff, not a leaderboard pick. MiniLM is attractive because it is small and fast, but the early Chronicle session history needed recall on short project-specific decisions where

the query and source chunk often do not use the same words. BGE-small worked better on Chronicle's own session data. One representative case was `polarity-fold-into-kind-match`: BGE returned the chunk containing `POLARITY_NONMATCH_FLOOR = 0.4`, while MiniLM put it outside the top-20. The main latency cost is carrying a local sentence-transformers model at all, especially on cold start, so the daemon matters.

**Separating spoken text from tool output.** File dumps and command output can drown out the discussion around them. The current chunker keeps spoken chunks clean, then indexes tool inputs and selected tool results as separate chunk kinds. That keeps code/output evidence available without letting it dominate every conversation window.

**Content-addressed IDs.** `chunk_id = sha256(session || first_turn || content)[:16]`; memory IDs use the same shape. Unchanged JSONLs can be skipped, and cache artifacts stay portable across eval workdirs.

# 5 The eval problem

There is not yet a public benchmark that directly measures Chronicle's shape: a coding agent using raw prior work sessions as memory while completing later tasks. The closest candidates test adjacent pieces.

MEMTRACK has the right general shape, but its main finding is also the problem Chronicle tries to bypass: agents often do not call memory tools. LoCoBench and LoCoBench-Agent test long-context software engineering, not a retrieval layer that tries to avoid a giant prefill. SWE-Bench variants are mostly single-task and single-session. LongMemEval and LOCOMO are multi-session, but conversational rather than coding-focused.

I found SWE-ContextBench later than I should have. It is a better neighbouring reference than SWE-Bench-CL because it explicitly studies context reuse in coding task sequences. It still is not the same measurement problem: Chronicle needs to test whether a hook-injected memory layer can recover useful project context from messy prior agent sessions.

That is why the eval harness is custom. It is not meant to prove that the benchmark design is ideal. Please complain about the benchmark quality: false positives, weak checks, convergent tasks, and unfair scenarios are all useful findings, because the evaluation design is part of what needs to improve. The harness exists because Chronicle needed a way to compare hook-on vs hook-off runs, preserve the source JSONLs, capture the injected context, and separate task completion from evidence that the agent used prior context.

A practical lesson from building the harness is that agent leakage is a real experimental threat. Early attempts without strong isolation made it too easy for runs to see traces, generated files, local state, or test setup files they should not have had. The current runner therefore executes each tuple inside a sandboxed Docker container with a per-tuple repository copy, isolated home/config paths, copied substrate, and taint checks for suspicious reads or writes. Building that isolation layer took a non-trivial amount of time, but it was necessary to make the comparison credible.

# 6 Eval structure

The harness is `chronicle_eval_v2/`. Earlier suites (`chronicle_eval/`, `evals/`) test deprecated markdown-only architectures and shouldn't be used as a model.

## 6.1 Variants

Headroom is the main comparison arm. I chose it because it was the most popular similar open-source framework I found: the GitHub repo has 1k+ stars. It is a separate memory tool with the opposite delivery style: an offline `learn -apply` step turns the session history into a curated `CLAUDE.md` plus `MEMORY.md`, and a local HTTP proxy on `127.0.0.1:8787` injects that summary into every model API call. Chronicle is per-prompt, lossy-but-fresh; Headroom is upfront, distilled-into-rules. The two delivery styles are opposites, which is why Chronicle vs. Headroom is a more informative comparison than Chronicle vs. nothing.

**Table 3:** Evaluation variants and setup.

| Variant | Setup |
|---|---|
| `chronicle-on` | Hook wired in `.claude/settings.json`; substrate copied into per-tuple workdir; conversation index pre-warmed |
| `baseline` | Stock `claude -p` with empty `settings.json`; no hook, no substrate |
| `headroom-on` | Comparison arm: Headroom's offline `learn -apply` writes `CLAUDE.md` + `MEMORY.md` and runs an HTTP proxy on `127.0.0.1:8787` |
| `stacked` | Chronicle hook plus Headroom proxy; tests whether the two substrates complement or collide |

The default CLI currently expands to all four variants unless `-variants chronicle-on baseline` is supplied. The two-arm `chronicle-on` vs `baseline` comparison is the intended headline, not the runner default.

## 6.2 Tracks

**Table 4:** Evaluation tracks.

| Track | Hook wired | Question |
|---|---|---|
| `controlled` | `UserPromptSubmit` | Does retrieval help on a tightly-scoped, single-source-session decision? |
| `dogfood` | `UserPromptSubmit` | Does retrieval help on real, noisy, multi-session Chronicle dev history? |
| `conversation` | `PreToolUse:Read/Grep/Glob` | Does mid-turn retrieval change read/search behaviour? |
| `vague` | `UserPromptSubmit` | Does retrieval help on open-ended user prompts mined from real sessions? |
| `behavior` | `UserPromptSubmit` | Does retrieval help the agent follow project-specific rules and conventions mined from sessions? |
| `integration` | `UserPromptSubmit` | Does retrieval help on multi-step refactors that compose decisions across many prior sessions? |

## 6.3 Per-scenario layout

**Listing 1:** Per-scenario directory layout.

```
scenarios/<track>/<scenario_id>/
- scenario.yaml
- claude_projects/<project>/<uuid>.jsonl
- repo/
- checks/*.sh
- calibration/ideal/
- calibration/noop/
```

The source JSONLs come from real project sessions, including fixed decoy sessions from the same project. That keeps the eval closer to production, where the target memory sits next to unrelated but plausible prior work.

## 6.4 Headline metrics

The two primary metrics are `hard_pass_rate` and `chronicle_signal_rate`. Hard pass is the floor: did the task get done at all? Signal is stricter than "Chronicle retrieved something." It asks whether the agent acted as if the right memory mattered: using the project-internal name, exact value, rejected alternative, or recipe in its behavior. Equal hard pass with higher signal means Chronicle changed *what* gets done, not *whether* it gets done.

**Table 5:** Headline evaluation metrics.

| Metric | What it captures |
|---|---|
| `hard_pass_rate` | Fraction of must-pass checks that passed; the correctness gate |
| `chronicle_signal_rate` | Fraction of "did the agent visibly use prior context" checks that passed; shows Chronicle-specific value |
| `total_cost_usd`, `duration_ms`, `num_turns` | Efficiency. Sometimes Chronicle wins by being faster at equivalent quality |
| `input_tokens`, `output_tokens`, `cache_read_tokens` | Token accounting |
| `hook_silent_failure_count` | Chronicle-on tuples where the hook produced no injection; flags indexing bugs or pathological `min_top_cosine` settings |
| `outer_writes` | Sandbox-escape detection: any Edit/Write outside the per-tuple `repo/` |
| `mean_search_calls_per_seed` | Conversation-track only; how often the agent invokes Read/Grep/Glob |

## 6.5 Scenario design

The main trap is the **convergent decision**: a task a strong model can solve from first principles. Useful scenarios make prior context matter:

- Project-internal names, such as `_safe_session_id` or `POLARITY_NONMATCH_FLOOR`.
- Multi-step recipes where the exact combination is project-specific.
- Tuning constants that are not obvious from current code.
- Local conventions that differ from common defaults.
- Rejected alternatives already discussed in prior sessions.

A separate calibration harness (`chronicle_eval_v2.calibrate`) runs check scripts against `ideal/` and `noop/` fixtures before the real run. It catches mechanical bugs (BSD-vs-GNU regex, bool-typo passing as truthy, false-positive on unchanged file) before they skew a real run.

## 6.6 Fairness controls

The harness tries to keep Chronicle wins tied to retrieval rather than leaked answers. Signal checks should require a concrete fact from prior sessions: a project-internal name, an exact value, a rejected alternative, or a multi-piece recipe. The starter repo and prompt should not hand the answer to the baseline. Each scenario is tested against `ideal/` and `noop/`, and long source sessions can use `source_cutoff_timestamp` so later eval-writing discussion does not leak into the substrate.

There are still rough edges. Some early vague-track wins were too lexical: the baseline answer was basically right but missed the exact token the check expected. Those cases should either be downgraded to ties or judged by a stronger rule: did the answer demonstrate access to a fact that only existed in prior session history? The docker sandbox and taint scanner also need manual review when path-name rules flag harmless reads inside the per-tuple home.

## 6.7 Data

The first completed track is `vague`: open-ended questions mined from real project sessions, run as paired `chronicle-on` vs. `baseline` tuples. Here, a *tuple* means one scenario, one seed, and one variant; a paired seed comparison contains two tuples, one for Chronicle and one for baseline. The original hard-pass check was intentionally weak for this track: it only verified that the agent wrote a non-trivial `answer.md`. That makes hard pass a completion metric, not a quality metric. I therefore manually regraded the traces for answer usefulness and prior-context use.

For the clean analysis below I removed any seed pair where either arm had a taint flag. After filtering, the data contains 62 clean paired seed comparisons, covering 24 scenarios. Nineteen seed pairs were dropped for taint.

**Table 6:** Manual regrading summary for clean vague-track pairs.

| Unit | Chronicle wins | Ties | Manual Chronicle score |
|---|---|---|---|
| Scenario-level | 14 | 10 | 0.792; bootstrap 95% CI [0.688, 0.896] |
| Seed-pair-level | 36 | 26 | 0.790; bootstrap 95% CI [0.726, 0.847] |

The main result is not that Chronicle makes agents write files. Both arms usually do that. The result is that, on clean vague prompts where prior session context matters, Chronicle is much more likely to answer with the right project-specific decision, previous failure, or integration detail.

### 6.7.1 Hard-pass-only cost

Cost is only meaningful for completed hard-pass tasks, so I calculated spend after filtering to hard-passed tuples. Chronicle completed 62 clean hard-pass tuples with recorded cost; baseline completed 60. Across completed hard-pass tuples, Chronicle's mean cost was $0.4201 and baseline's was $0.3818.

The paired cost delta is the stricter comparison: only seed pairs where both arms passed hard checks and both costs were recorded. On those 60 completed pairs, Chronicle's mean cost was $0.3891, baseline's mean cost was $0.3818, and the mean paired delta was +$0.0073 per completed task. The median paired delta was -$0.0471, and the bootstrap 95% CI for the mean delta was [-$0.0642, +$0.0947].

Average wall time on successful hard-pass tuples was 118.4 seconds for baseline and 145.0 seconds for Chronicle. The paired comparison is stricter: among the 60 clean seed pairs where both arms passed the hard check, baseline averaged 118.4 seconds and Chronicle averaged 133.5 seconds. The mean paired wall-time delta was +15.0 seconds, the median paired delta was +4.9 seconds, and the bootstrap 95% CI was [-2.1, +33.5] seconds.

**Table 7:** Completed-task cost and wall-time summary for the vague track.

| Metric | Baseline | Chronicle | Delta | Notes |
|---|---|---|---|---|
| Completed hard-pass tuples with cost | 60 | 62 | +2 | Chronicle completed two additional clean tuples with recorded cost |
| Mean cost over completed hard-pass tuples | $0.3818 | $0.4201 | +$0.0384 | Per-arm completed-task means |
| Mean paired cost, both arms completed | $0.3818 | $0.3891 | +$0.0073 | 60 paired completed comparisons |
| Median paired cost delta | – | – | -$0.0471 | Chronicle was cheaper in the median completed pair |
| Bootstrap 95% CI for paired cost delta | – | – | [-$0.0642, +$0.0947] | Cost delta is noisy and crosses zero |
| Mean wall time, successful hard-pass tuples | 118.4s | 145.0s | +26.6s | Per-arm completed-task means |
| Mean paired wall time, both arms completed | 118.4s | 133.5s | +15.0s | 60 paired completed comparisons |
| Median paired wall-time delta | – | – | +4.9s | Chronicle was slightly slower in the median completed pair |
| Bootstrap 95% CI for wall-time delta | – | – | [-2.1s, +33.5s] | Wall-time delta is noisy and crosses zero |

So the efficiency result is modest: Chronicle buys a large quality/usefulness gain with no clear completed-task cost penalty in this run. Mean paired cost is slightly higher, median paired cost is lower, and the confidence interval crosses zero. Wall time shows the same shape: Chronicle is slightly slower on average, but the uncertainty interval crosses zero.

### 6.7.2 Behavior and integration

I then ran the same paired treatment on the completed `behavior` and `integration` tracks, now with all three variants: `chronicle-on`, `headroom-on`, and `baseline`. For these tracks, taint flags are real experimental contamination rather than harmless path-name artifacts, so I removed every tuple with a `taint_kinds` flag before

analysis. That removes five behavior tuples and zero integration tuples. Pairwise comparisons are formed at the `(scenario, seed)` level and include a pair only when both relevant variants are present and untainted.

For signal, I use the tuple-level signal pass from the harness: all Chronicle-signal checks for that tuple must pass. That is stricter than partial credit over individual signal checks. For cost and wall time, I use the same completed-task denominator as above: a paired cost/time comparison is included only when both variants passed the hard check and both have recorded cost or duration. No completed untainted hard-pass tuple was missing cost or duration.

**Behavior.** Chronicle completed all untainted behavior tuples and showed a large signal lift over both baseline and Headroom. The cost result is more modest: Chronicle's per-arm completed-task mean is higher, but the paired completed-task cost CI crosses zero against both comparison arms.

**Table 8:** Behavior-track aggregate results after taint filtering.

| Variant | N | Hard pass | Signal pass | Cost N | Mean cost | Wall N | Mean wall |
|---|---|---|---|---|---|---|---|
| Baseline | 99 | 92/99 (92.9%) | 51/99 (51.5%) | 92 | $0.0709 | 92 | 55.7s |
| Chronicle | 100 | 100/100 (100.0%) | 81/100 (81.0%) | 100 | $0.0866 | 100 | 69.3s |
| Headroom | 102 | 96/102 (94.1%) | 60/102 (58.8%) | 96 | $0.0760 | 96 | 72.0s |

**Table 9:** Behavior-track paired quality deltas.

| Comparison | Metric | N | Mean diff. | Median diff. | 95% CI | Width |
|---|---|---|---|---|---|---|
| Chronicle–Baseline | Hard pass | 98 | +0.061 | 0.000 | [0.020, 0.112] | 0.092 |
| Chronicle–Baseline | Signal | 98 | +0.296 | 0.000 | [0.204, 0.388] | 0.184 |
| Headroom–Baseline | Hard pass | 99 | +0.010 | 0.000 | [-0.051, +0.071] | 0.121 |
| Headroom–Baseline | Signal | 99 | +0.071 | 0.000 | [-0.030, +0.172] | 0.202 |
| Chronicle–Headroom | Hard pass | 100 | +0.060 | 0.000 | [0.020, 0.110] | 0.090 |
| Chronicle–Headroom | Signal | 100 | +0.230 | 0.000 | [0.130, 0.330] | 0.200 |

**Table 10:** Behavior-track paired efficiency deltas among completed hard-pass pairs.

| Comparison | Metric | N | Mean diff. | Median diff. | 95% CI | Width |
|---|---|---|---|---|---|---|
| Chronicle–Baseline | Cost | 92 | +$0.0121 | +$0.0014 | [-$0.0031, +$0.0300] | $0.0331 |
| Chronicle–Baseline | Wall time | 92 | +14.1s | +9.3s | [+7.2s, +22.2s] | 15.0s |
| Headroom–Baseline | Cost | 88 | +$0.0048 | +$0.0111 | [-$0.0058, +$0.0141] | $0.0199 |
| Headroom–Baseline | Wall time | 88 | +15.0s | +28.7s | [+2.5s, +27.1s] | 24.6s |
| Chronicle–Headroom | Cost | 94 | +$0.0112 | -$0.0040 | [-$0.0030, +$0.0285] | $0.0315 |
| Chronicle–Headroom | Wall time | 94 | -0.9s | -18.2s | [-12.0s, +10.8s] | 22.8s |

The behavior result is strongest on correctness and signal. Chronicle's hard-pass lift over baseline is +6.1 percentage points with bootstrap 95% CI [2.0, 11.2] points. Its signal lift over baseline is +29.6 points, CI [20.4, 38.8]. Against Headroom, Chronicle is also ahead on both completion and signal. The cost penalty is not resolved by this run because the paired completed-task cost intervals cross zero; wall time against baseline is slower, while wall time against Headroom crosses zero.

**Integration.** Integration is smaller, so the confidence intervals are much wider. Chronicle hard-passed every untainted integration tuple and had the highest signal rate, but several pairwise intervals cross zero and should be read as directional rather than conclusive.

**Table 11:** Integration-track aggregate results after taint filtering.

| Variant | N | Hard pass | Signal pass | Cost N | Mean cost | Wall N | Mean wall |
|---|---|---|---|---|---|---|---|
| Baseline | 12 | 10/12 (83.3%) | 4/12 (33.3%) | 10 | $0.5393 | 10 | 269.0s |
| Chronicle | 12 | 12/12 (100.0%) | 9/12 (75.0%) | 12 | $0.4832 | 12 | 263.4s |
| Headroom | 12 | 9/12 (75.0%) | 6/12 (50.0%) | 9 | $0.5276 | 9 | 247.9s |

**Table 12:** Integration-track paired quality deltas.

| Comparison | Metric | N | Mean diff. | Median diff. | 95% CI | Width |
|---|---|---|---|---|---|---|
| Chronicle–Baseline | Hard pass | 12 | +0.167 | 0.000 | [0.000, 0.417] | 0.417 |
| Chronicle–Baseline | Signal | 12 | +0.417 | +1.000 | [-0.083, +0.833] | 0.917 |
| Headroom–Baseline | Hard pass | 12 | -0.083 | 0.000 | [-0.250, 0.000] | 0.250 |
| Headroom–Baseline | Signal | 12 | +0.167 | 0.000 | [-0.167, +0.500] | 0.667 |
| Chronicle–Headroom | Hard pass | 12 | +0.250 | 0.000 | [0.000, 0.500] | 0.500 |
| Chronicle–Headroom | Signal | 12 | +0.250 | +0.500 | [-0.250, +0.667] | 0.917 |

**Table 13:** Integration-track paired efficiency deltas among completed hard-pass pairs.

| Comparison | Metric | N | Mean diff. | Median diff. | 95% CI | Width |
|---|---|---|---|---|---|---|
| Chronicle–Baseline | Cost | 10 | -$0.0051 | -$0.0242 | [-$0.1275, +$0.1400] | $0.2676 |
| Chronicle–Baseline | Wall time | 10 | +17.3s | -8.4s | [-39.3s, +79.2s] | 118.5s |
| Headroom–Baseline | Cost | 9 | -$0.0475 | -$0.0250 | [-$0.1472, +$0.0489] | $0.1961 |
| Headroom–Baseline | Wall time | 9 | -36.0s | -20.8s | [-91.2s, +19.5s] | 110.7s |
| Chronicle–Headroom | Cost | 9 | +$0.0446 | -$0.0453 | [-$0.0503, +$0.1551] | $0.2054 |
| Chronicle–Headroom | Wall time | 9 | +56.0s | +24.3s | [+7.3s, +111.1s] | 103.7s |

The integration result should not be overclaimed. Chronicle's completion and signal means are higher, but the paired signal CIs are wide because there are only 12 seed pairs. Cost is indistinguishable from zero in all pairwise integration comparisons. Wall time is especially noisy; the only clearly positive wall-time interval is Chronicle versus Headroom among the nine pairs where both completed.

Chronicle did not lose any hard-pass comparison to baseline in the untainted behavior or integration data. It did lose signal checks. Against baseline, the untainted losses are `fp-os-blocker-until-secrets` seed 0 on behavior, where the agent completed the task but did not verify or acknowledge the secrets blocker, and `feature-from-pattern` seeds 0, 1, and 2 on integration, where the agent completed the task but did not satisfy the prior component-pattern check. Against Headroom, the clean signal losses are concentrated in procedural and pattern scenarios: `fp-os-cost-budget`, `fp-os-discussion-shape`, `fp-os-granola-url-format`, `rubric-headroom-package-manager-ritual`, and `feature-from-pattern`.

These losses match the improvement plan rather than contradicting it. I am leaving them in the table because they are useful failures: not mysterious benchmark noise, but small, legible gaps in the current delivery surface. `fp-os-blocker-until-secrets` is the "retrieval injects advice but does not enforce it" failure: the fix is to promote mined blockers and proscriptions into the PreToolUse/Bash guard so active prerequisites can become `warn`, `ask`, or `deny` controls instead of passive context. The Headroom-shaped losses are procedural-memory failures: repeated token prefixes, package-manager rituals, budget constraints, URL/path conventions, and component patterns. The plan's answer is behavioral/tool-result indexing, frequency-aware convention and ritual mining, stable procedural slots, and contextual chunk prefixing: adopt Headroom's aggregate-by-frequency advantage while keeping Chronicle's fresher per-prompt retrieval.

## 6.8 Example traces

*Because the test data came from my own code sessions, I have slightly tweaked the naming of the prompts and the logic flow after the fact. The actual evals were run with the real data.*

**Integration replacement status, seed 0.** The prompt asked what happened to a partially completed replacement of an older third-party integration with a newer provider. Both runs were clean and both passed the hard completion check. Baseline cost $0.2812; Chronicle cost $0.1359.

Baseline answered from the current repository. It concluded that the newer provider had been mostly wired up, that dead code from the older provider remained, and that a current API error was blocking ship. That is a plausible repo-only answer, but it misses the user's actual historical question: what happened to the replacement work?

Chronicle recovered the prior-session fact: the replacement edits had only existed in an uncommitted working tree and were later overwritten by a reset-style restore of one backend function file. It also identified that the only surviving record was a diff captured in the earlier conversation, so the work needed to be reconstructed rather

than merely found in the current repo. This is exactly the kind of answer the baseline cannot infer from the current source tree alone.

**Document-link sync status, seed 0.** The prompt asked whether a daily CRM sync populated links to company documents. Both runs were clean and both passed the hard completion check. Baseline cost $0.1371; Chronicle cost $0.1163.

Baseline searched the current codebase and concluded that document links did not exist anywhere in the repo. That answer is locally consistent: the current files did not contain the expected field names or mappings.

Chronicle answered the current-code question but added the missing historical context: a prior document-sync job had been the planned source of truth, matching cloud-drive files to companies, storing the matched links in a dedicated table, and patching the CRM field. It also recalled a later decision for the daily sync to read manually-set links back from the CRM and store synthetic link rows keyed to the CRM record. The baseline saw absence; Chronicle explained both the absence and the intended design.

# 7 Current limitations

Chronicle has only just started, but it has reached basic operational ability: it can index real sessions, inject context, run evals, and produce useful run logs and metrics. The remaining work is about making that reliable enough to use without babysitting it.

## 7.1 Implementation

- The `compose.enabled` path does not yet refresh JSONLs incrementally.
- `USER_AUTHORED_BOOST` is defined but not wired into scoring. Citation prior scores exist, but writeback is opt-in and needs more validation.
- Pinned policy can be shown early on `UserPromptSubmit`, but it is not yet a full `PreToolUse` hard block.
- Action mining works for conventions, rituals, and things not to do, but broader detection coverage is still partial.

## 7.2 Architecture

- Dense retrieval can still mix up negation, time, and replaced claims.
- Pre-turn retrieval can miss because the model has not reasoned about the task yet.
- Several keyword lists assume English and common HTTP/POSIX error wording.
- The SessionStart cache benefit is plausible but not measured yet.
- Large-project cost comes mainly from typed-claim LLM extraction, not local chunk indexing.

## 7.3 Eval harness

- The runner defaults to four variants, while the headline comparison is usually two-arm.
- Calibration coverage is incomplete.
- Prewarm fingerprint omits `source_cutoff_timestamp`, so cutoff changes can reuse stale/leaky caches.
- Hook logs and WIN/TIE/LOSE classification need stricter correctness gates.
- The taint scanner does not cover every host-transcript leak path yet.

Separately, the repo isn't yet open-source clean and the package shape is currently a research-monorepo rather than a clean public package.

# 8 Further developments

## 8.1 Build out the current use case

The near-term goal is to make Chronicle reliable as a day-to-day Claude Code memory layer. The practical work is clear: make `compose.enabled` the default route by refreshing the conversation index there, connect user-authored claims to ranking, turn pinned rules into `PreToolUse` blocks, keep improving action mining, measure whether the

SessionStart prelude reduces cost, make English-specific keyword lists work beyond English, and clean up the package shape.

The other immediate issue is cold start. BGE-small is fast once loaded, but sentence-transformers cold load can cost several seconds, and repeated hook fires turn that into visible delay. The intended fix is a persistent daemon that loads the embedder once, serves hook-time embedding/search over a local socket, and falls back to in-process embedding when unavailable. The daemon path exists, but its background mining sweep is not reliable yet, so the honest near-term goal is to use it first for latency and then harden it as a cache producer.

## 8.2 Measuring memory over large coding systems

SWE-ContextBench is the closest recent pointer. Zhu, Hu, and Wu build related coding task sequences and measure prediction accuracy, time, and cost under different context-reuse settings. Their result maps cleanly onto Chronicle's problem: selected summaries help, while unfiltered or poorly selected context can hurt.

The next step is a system that measures memory recall across large, ongoing codebases: real session streams, task N evaluated against sessions 1..N-1, and checks that ask whether the agent used the remembered project context. Contained single-task evals are useful, but they are not the best model for where agentic programming seems to be heading: longer projects, recurring agents, and memory that carries across work.

## 8.3 Petri-style memory audits

Anthropic's Petri framework (Parallel Exploration Tool for Risky Interactions) is an alignment-auditing tool, not a coding-memory benchmark, but its harness shape is what is relevant here. Petri runs an auditor agent through realistic multi-turn scenarios, lets it interact with a target model and simulated tools, and then has an LLM judge score the resulting transcripts across a rubric of behavioral dimensions. Petri 2.0's headline change is making it less obvious that the model is being evaluated: making scenarios feel like real deployment rather than tests, alongside an expanded scenario library covering more behaviors and contexts.

A Chronicle version would borrow that shape in two layers. A scenario generator would seed each scenario with the memory challenge under test: stale memories, missing prior decisions, misleading near-matches, or retrieved chunks the agent *should* ignore. A Petri-style auditor would then drive the multi-turn coding interaction against a target agent with Chronicle on or off. The judge would score not just final correctness, but whether the agent relied on the right memory, ignored bad memory, or wasted turns re-discovering context that should have been shown. That complements the current scripted check harness rather than replacing it.

## 8.4 Do citations reflect what the agent used?

One useful distinction in recent RAG citation work is citation correctness versus citation faithfulness: a cited source can support a claim even if the model did not rely on it. That work reports that up to **57% of citations lack faithfulness** in standard RAG QA.

Chronicle can test a smaller, practical version of that question in coding. It can join `injections.jsonl`, `citations.jsonl`, file edits, and Bash calls to ask: when memory was injected and cited, did the agent's action actually depend on it? If unfaithful citations predict failed checks, citation faithfulness becomes a useful signal before a run finishes rather than an after-the-fact paper metric.